Deployment of a Cloud-Based Web Application using AWS EC2 and RDS

Submitted by:

Mahesh

College:

PSCMR College of Engineering and Technology

Course:

Cloud Computing / AWS Lab

Objective:

To deploy a cloud-based web application using AWS EC2 and connect it with Amazon RDS database.

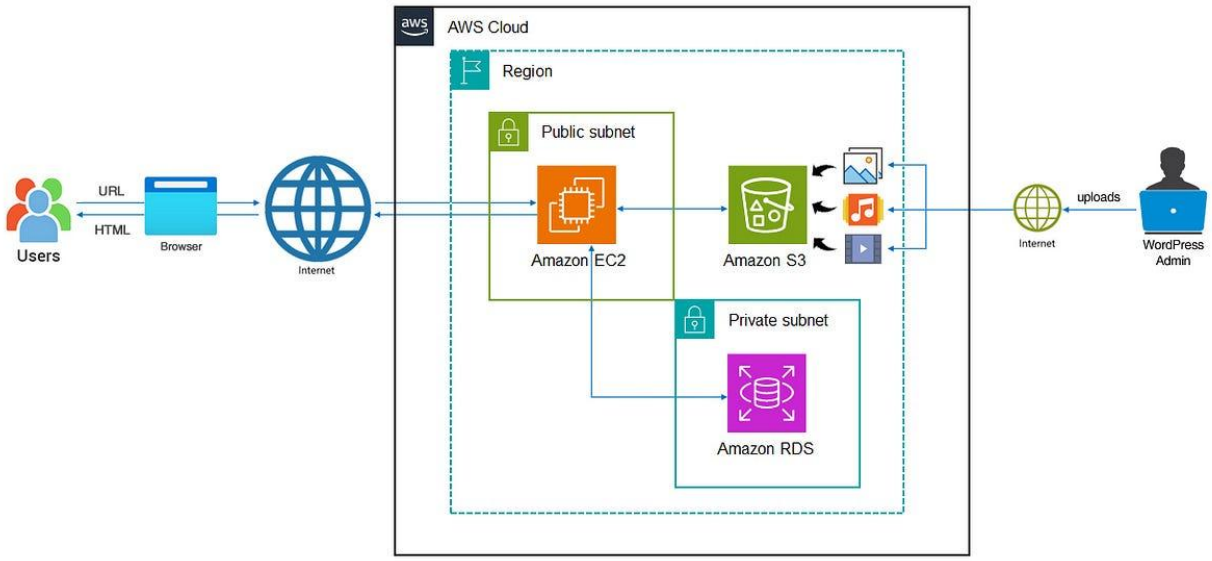
Introduction:

Cloud computing provides on-demand access to computing resources such as servers, storage, and databases. Amazon Web Services (AWS) is a leading cloud platform that offers scalable and reliable services.

In this project, an EC2 instance is used to host a WordPress application, and Amazon RDS is used as the backend database.

Architecture Description:

The user accesses the application through a web browser. The request is sent over the internet to the EC2 instance, where Apache and WordPress are hosted. The EC2 instance communicates with the Amazon RDS database to store and retrieve data.

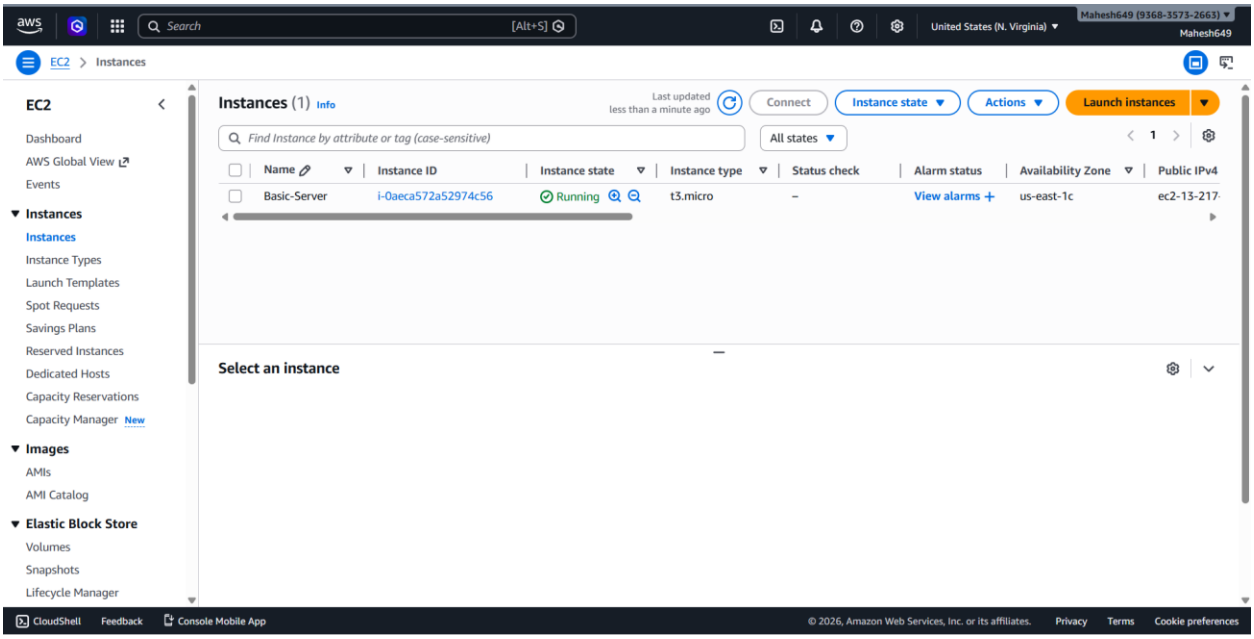


Step 1: Launch EC2 Instance

An Amazon EC2 instance was launched to host the web application. The instance was created using Amazon Linux and configured with necessary settings such as instance type, key pair, and security group.

The instance is currently in **running state**, which indicates that the server is active and ready to accept connections.

The instance type used is **t3.micro**, which is suitable for lightweight applications and falls under the free tier.



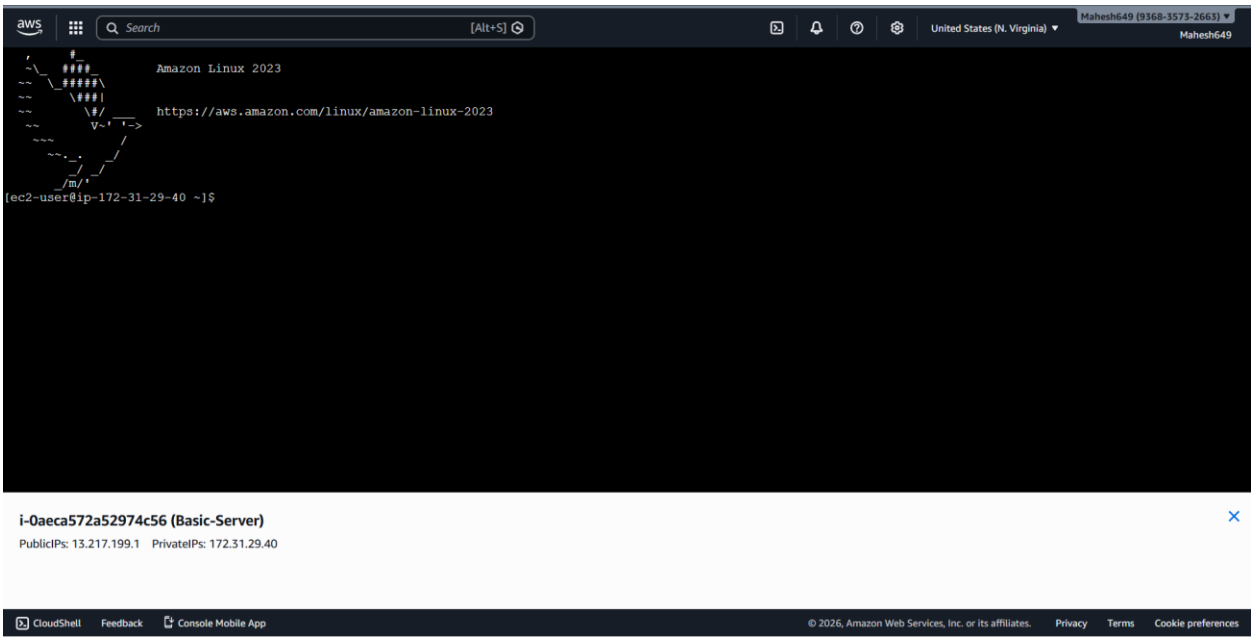
The EC2 instance acts as a virtual server in the cloud where the web application will be deployed. It provides computing resources such as CPU, memory, and storage.

Step 2: Connect to EC2 using SSH

After launching the EC2 instance, the server was accessed using **SSH (Secure Shell)**. SSH allows secure remote login to the virtual machine.

The connection was established using the EC2 instance's public IP address and key pair authentication.

Once connected, the terminal displays the Amazon Linux environment, confirming successful access to the EC2 instance.



SSH ensures secure communication between the local system and the EC2 instance using encryption.

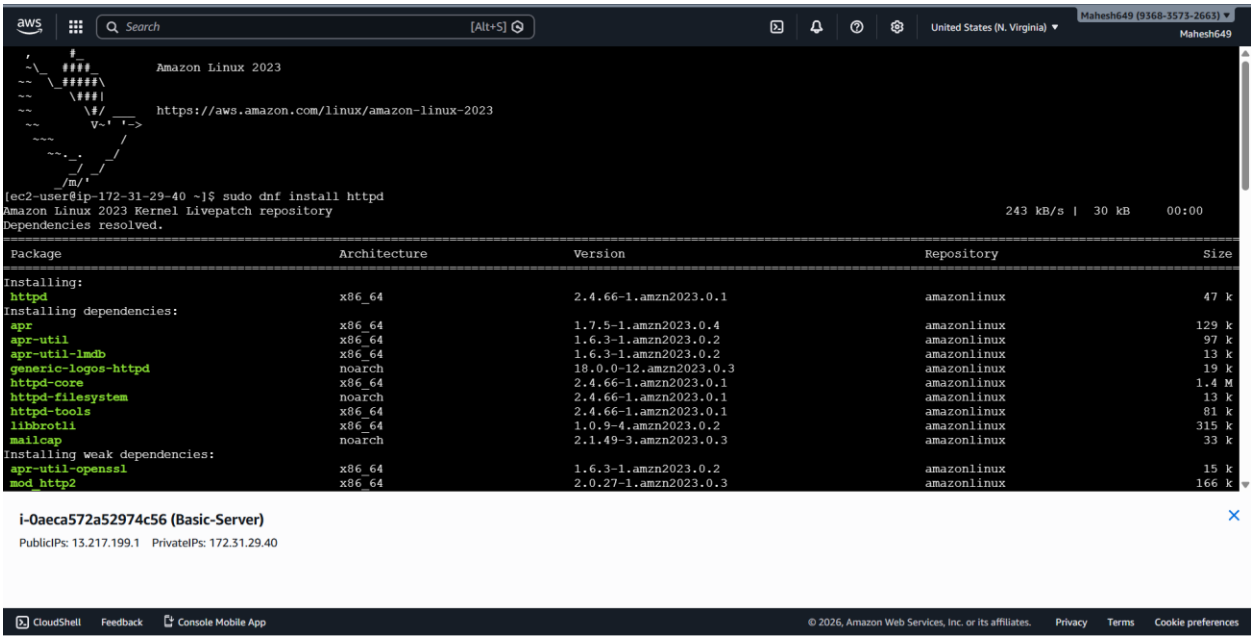
Step 3: Install Apache Web Server

After successfully connecting to the EC2 instance, the Apache web server was installed. Apache is used to host web applications and serve web pages to users over the internet.

The installation was performed using the following command:

• sudo dnf install httpd

This command installs the Apache HTTP server along with its required dependencies.



After installation, Apache can be started using:

• sudo systemctl start httpd
• sudo systemctl enable httpd

• start → starts the web server

• enable → ensures it runs automatically after reboot

Apache acts as a web server that handles incoming HTTP requests and delivers web pages to users.

Step 4: Hosting a Web Page on EC2 Instance

After installing and starting the Apache web server, a simple web page was created and hosted on the EC2 instance.

The default web directory used by Apache is:

/var/www/html/

A sample HTML file was created inside this directory to verify the web server functionality.

Example command used:

sudo nano /var/www/html/index.html

The web page was successfully accessed using the public IP address of the EC2 instance:

http://13.217.199.1

This confirms that:

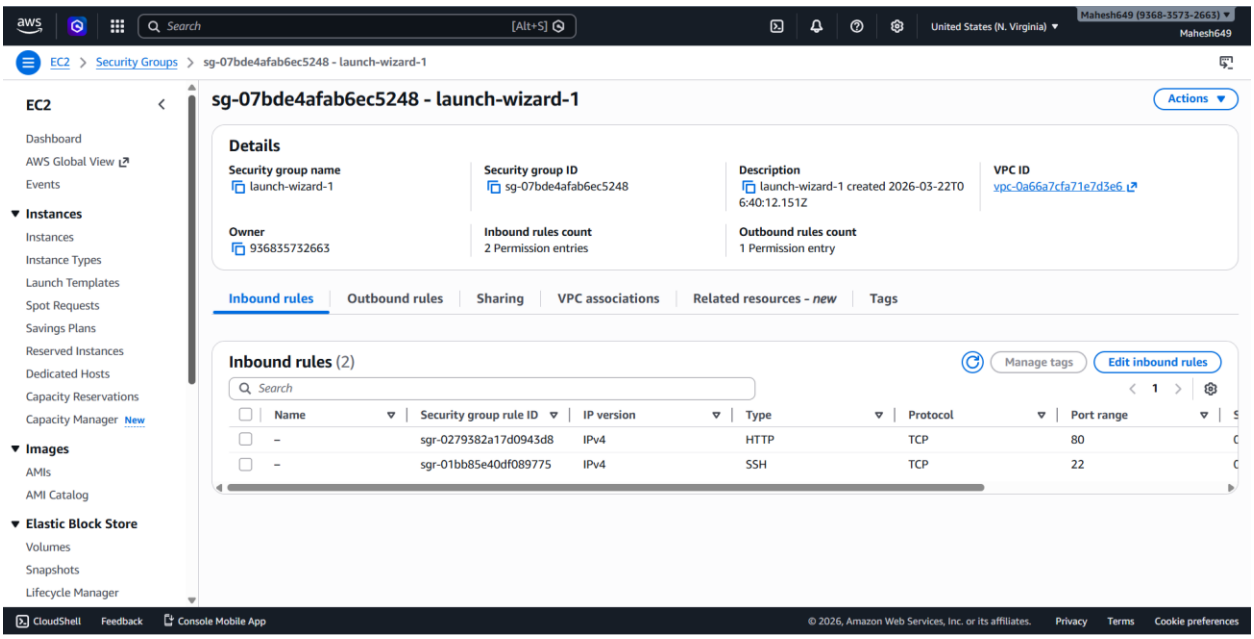
- The EC2 instance is running
- Apache web server is active

- Security Group allows HTTP (Port 80)
- Web application is successfully deployed

Step 5: Configure Security Group

To allow access to the web application and enable remote connection, Security Group rules were configured.

A Security Group acts as a virtual firewall that controls inbound and outbound traffic to the EC2 instance.



The following inbound rules were configured:

- **HTTP (Port 80)** → Allows users to access the website via browser
- **SSH (Port 22)** → Allows secure remote login to EC2 instance

Protocol used: **TCP**

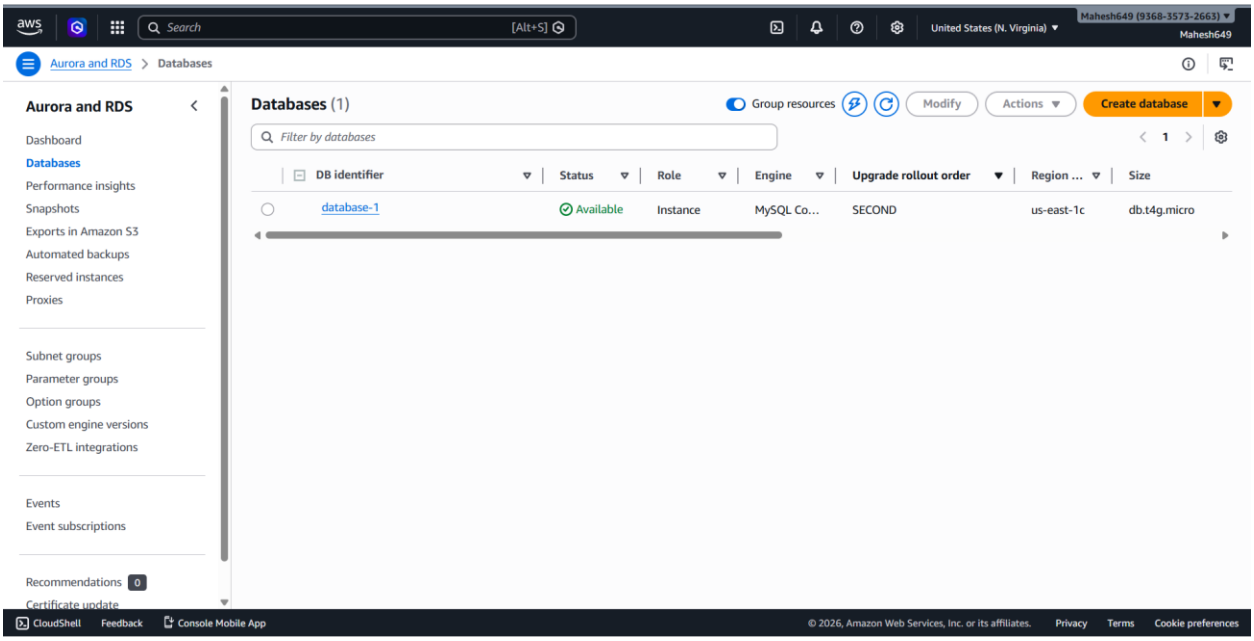
Without enabling HTTP (Port 80), the hosted web application cannot be accessed through a browser

Security Groups provide instance-level security and follow a stateful filtering mechanism, meaning return traffic is automatically allowed.

Step 6: Configure Amazon RDS (MySQL Database)

To store application data, Amazon RDS (Relational Database Service) was used. A MySQL database instance was created and configured.

Amazon RDS simplifies database management by handling backups, updates, and scaling automatically.



The following configuration was used:

- **DB Instance Name:** database-1
- **Engine:** MySQL
- **Instance Type:** db.t4g.micro
- **Status:** Available

The EC2 instance can connect to the RDS database using:

- Endpoint (from RDS dashboard)
- Username and password
- Port: 3306 (MySQL default)

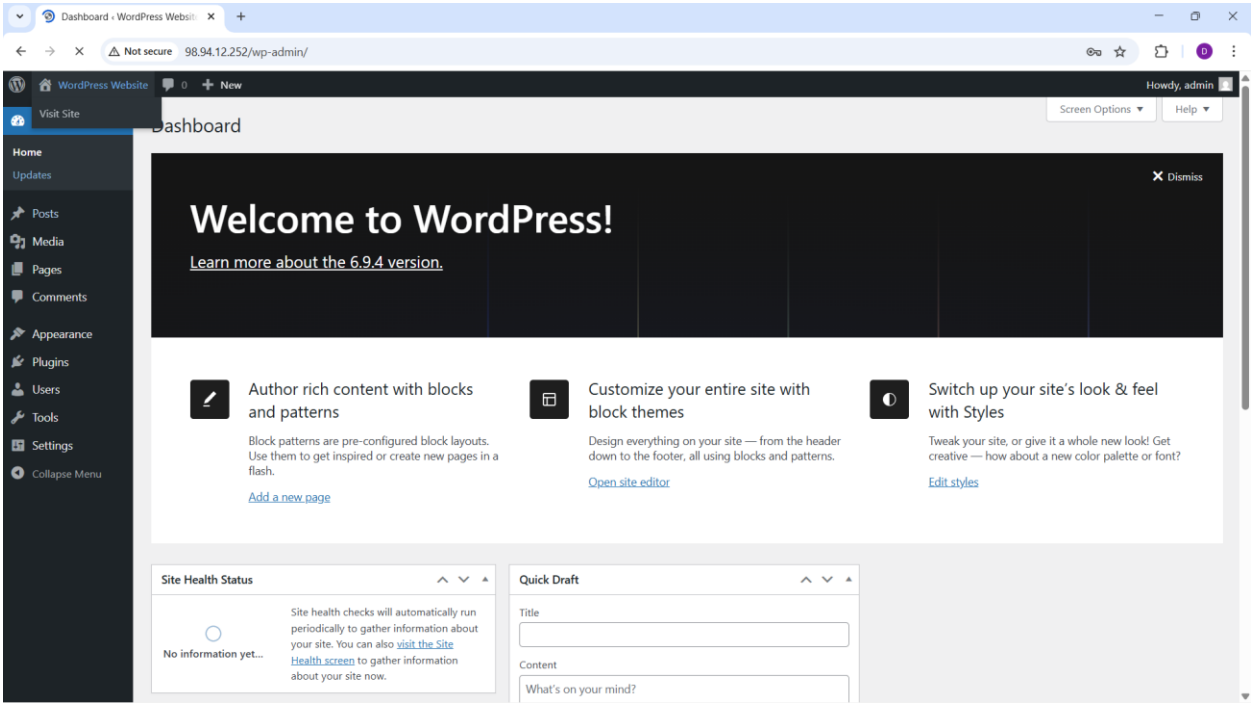
Amazon RDS provides a fully managed database service, reducing the need for manual database administration.

Step 7: WordPress Installation and Admin Access

After configuring the EC2 instance and connecting it with the RDS database, WordPress was successfully installed.

The WordPress admin dashboard was accessed using:

http://<Public-IP>/wp-admin



The dashboard allows administrators to:

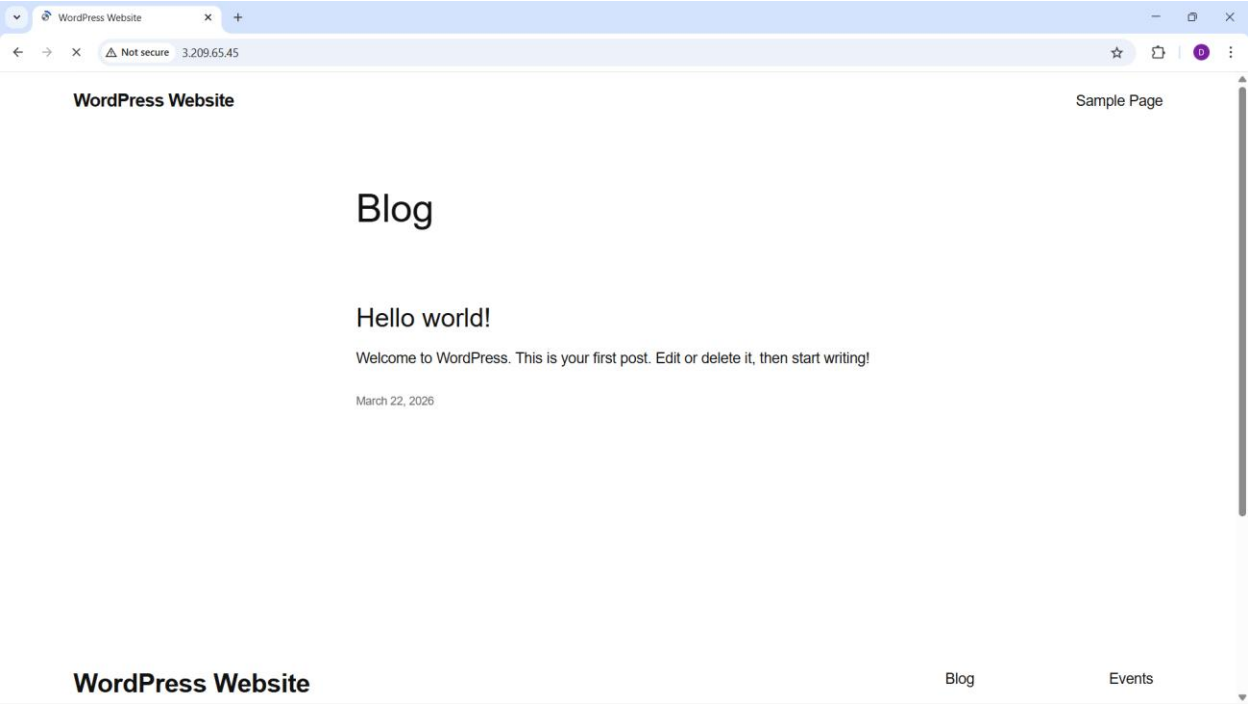
- Create and manage posts and pages
- Install themes and plugins
- Configure website settings

This confirms that the WordPress application is successfully installed and running on the EC2 instance.

Step 8: Elastic IP Association and Final Output

To ensure a stable public IP address, an Elastic IP was allocated and associated with the EC2 instance.

After associating the Elastic IP, the WordPress website was accessed using the new public IP address.



Elastic IP ensures:

- Static IP address (does not change after restart)
- Reliable access to the web application
- Improved availability for production use

The successful loading of the WordPress website confirms that the EC2 instance, Apache server, and RDS database are correctly integrated.

CONCLUSION

The project successfully demonstrates the deployment of a cloud-based web application using AWS services.

Key achievements:

- EC2 instance setup and configuration
- Apache web server deployment
- WordPress application hosting
- Security Group configuration
- Amazon RDS integration
- Elastic IP association

The application was successfully accessed through the browser, proving that all components are correctly integrated.

This project highlights practical implementation of cloud computing concepts such as Infrastructure as a Service (IaaS), web hosting, and managed database services.